



Quick Summary of Mobile Robots

Prof. V. Muthukumar





Motion Modeling

- *Kinematic model* describes
 - geometric relationships that are present in the system - The effect of a robot's geometry on its motion.
 - relationship between input (control) parameters and behavior of a system in the system - **If the motors move this much, where will the robot be?**
 - Expressed as first order differential equations
- *Dynamic models* describes
 - system motion when forces are applied to the system - The effect of all forces (internal and external) on a robot's motion.
 - physics of motion where forces, energies, system mass, inertia, and velocity parameters - **If the motors apply this much force, where will the robot be?**
 - Expressed as second order differential equations





Mobile Robot Kinematics

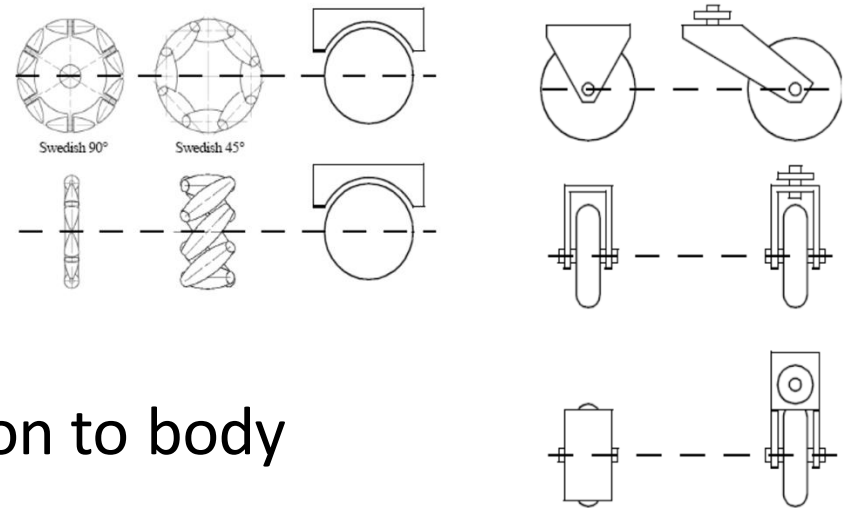
- Kinematics is the most basic study of how mechanical systems behave.
- we need to understand the mechanical behavior of the robot both
 - to design appropriate mobile robots for tasks and
 - to understand how to create control software for an instance of mobile robot hardware.





Wheels and their constraints

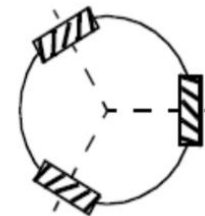
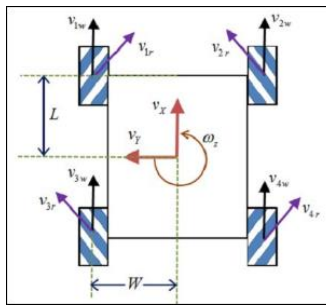
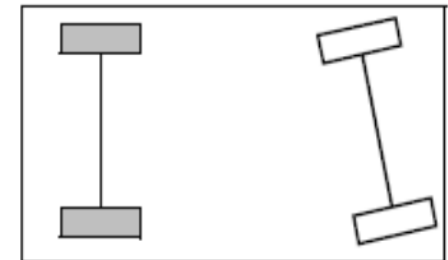
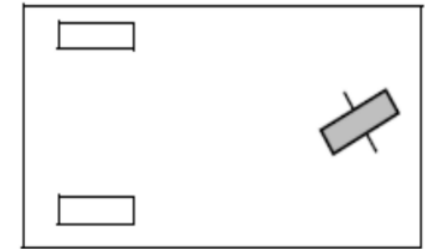
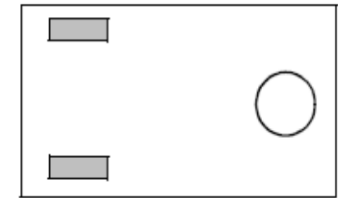
- Regular wheels
 - Two degrees of freedom; rotation around the (motorized) wheel axle and the contact point
 - Non-holonomic constraint
 - $\dot{x}=v, \dot{y}=0$
- Omnidirectional wheel
 - Mecanum/Swedish wheel (45)
 - omni or poly wheel (90)
 - can rotate and slide laterally
 - No constraints
- Based on mounting/suspension to body
 - Castor wheel
 - Castor wheel: Three degrees of freedom; rotation around the wheel axle, the contact point and the castor axle.
 - Caster Ball: No constraints. Just support.





Characteristics of a Wheeled Robot

- Stability of a vehicle is be guaranteed with 3 wheels
 - Combining actuation and steering on one wheel makes the design complex and adds additional errors for odometry.
- Stability is improved by 4 and more wheel
 - Ackerman steering
- Most arrangements are non-holonomic - require high control effort



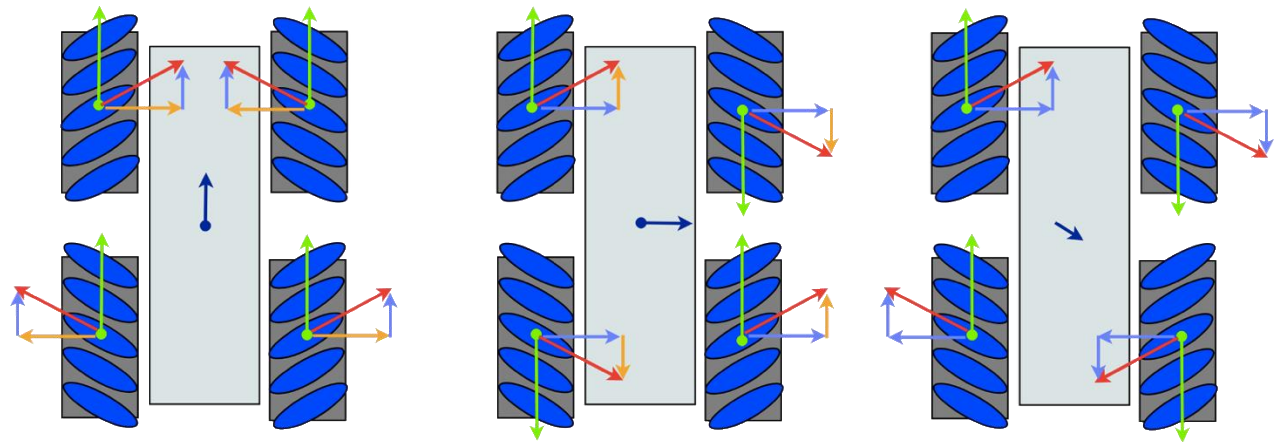
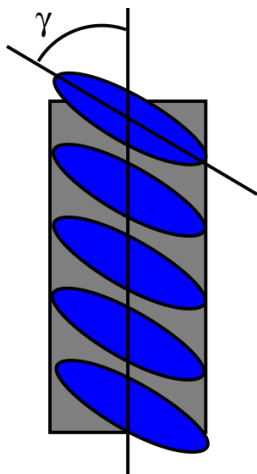
Omnidirectional Drive





Wheels in Detail

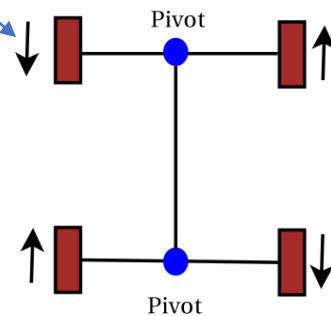
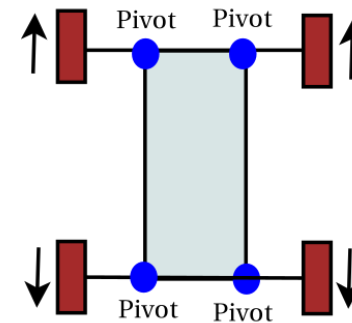
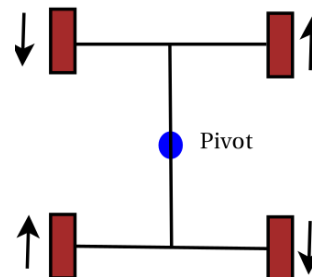
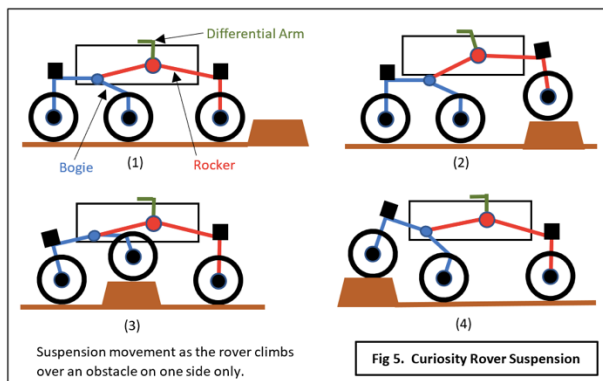
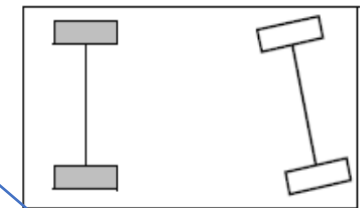
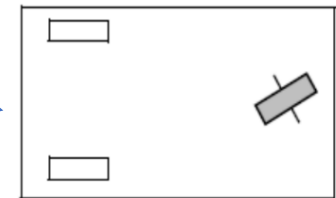
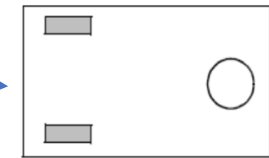
- effectiveness of the wheel is given by the contact area of the wheel (width, tread), friction.
- Maneuverability by steering control or traction control.
- Mecanum wheel have less ground friction in a turn in comparison to a skid steer/DD requiring much less torque.





Steering Models

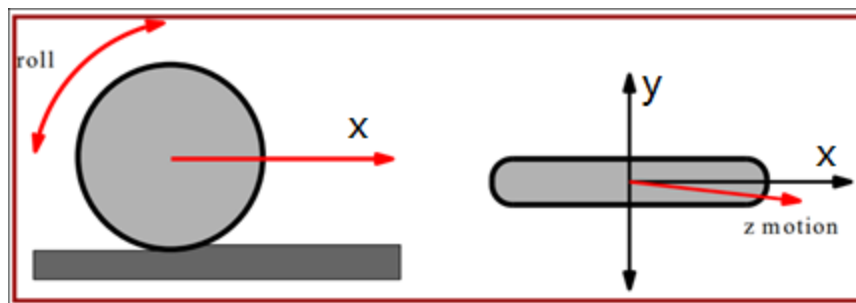
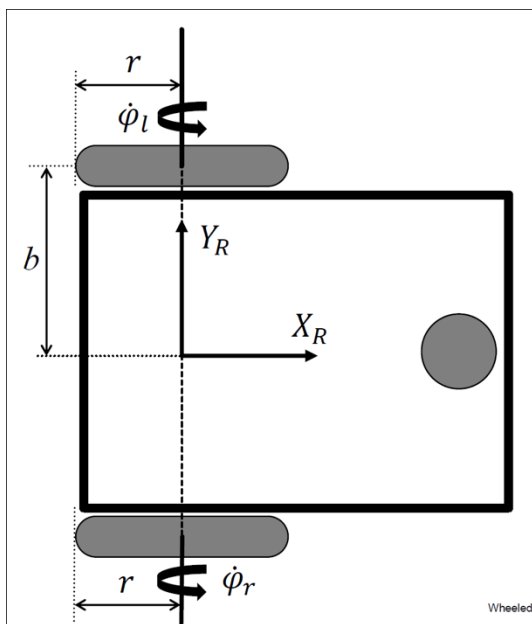
- Unicycle Model
- Differential Drive Model
- Tricycle Model
- Ackermann Model
- Dual Differential Drive (DDD)
- Four Wheel Steer (FWS)
- Articulated Drive (AD)





Wheel kinematic constraints

- express constraints on the motions of individual wheels.
- motions of individual wheels can later be combined to compute the motion of the robot as a whole.
- Assumptions:
 - wheel always remains vertical and that there is in all cases one single point of contact between the wheel and the ground plane.
 - Each wheel in WMR can freely rotate around its own axis.
 - there is no sliding at this single point of contact.



$$\text{Forward velocity: } \dot{x} = r \frac{(\dot{\phi}_r + \dot{\phi}_l)}{2}$$

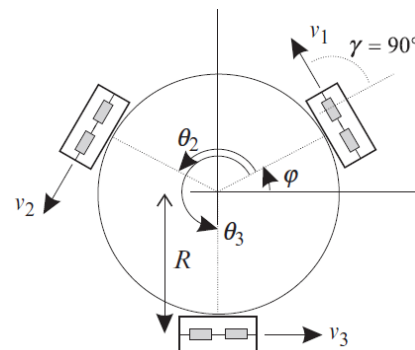
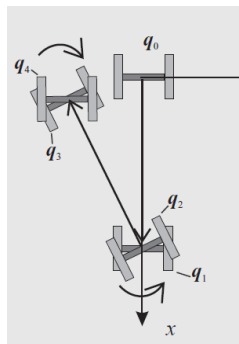
$$\text{No-sliding: } \dot{y} = 0$$





Nonholonomic Vs Holonomic

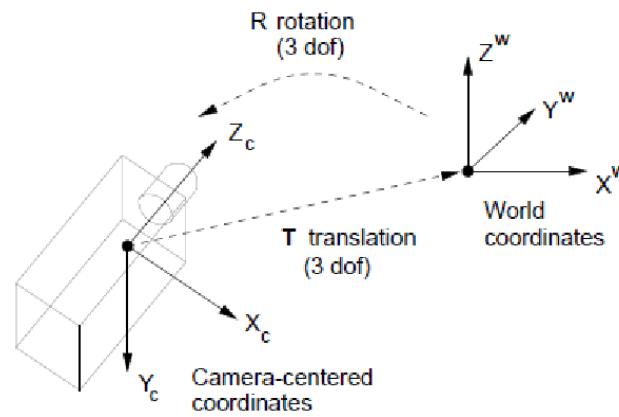
- a robot is **nonholonomic** if it cannot move to change its pose instantaneously in all available directions.
 - makes it more difficult to navigate between two arbitrary points
 - need to resort to multiple moves (parallel parking)
- a robot is **holonomic** if it can move to change its pose instantaneously in all available directions.
 - robot's differential motion is unconstrained





Rigid Body Motion

- Rigid body
 - one in which all interparticle distances are fixed by internal forces of constraint.
 - ignores elastic and plastic deformations to which any real body is susceptible
- Rigid Body Motion
 - translations of the body as a whole,
 - rotations of the body about some fixed axis
 - reduces the problem to one with six degrees of freedom





Summary of Kinematic Models and Constraints

- Deriving a model for the whole robot's motion is a bottom-up process
- Each individual wheel contributes to the robot's motion and, at the same time, imposes constraints on robot motion.
- Wheels are tied together based on robot chassis geometry, and therefore their constraints combine to form constraints on the overall motion of the robot chassis.
- Usually, we assume the robot body is rigid body system.
- Assume robot mass remains constant and the center of mass is the center of body and remains constant.





Notes to me

- **Perseverance (Mars 2020)**: This rover landed on Mars in 2021 and is on a mission to collect and cache Martian rocks and sediment samples for future return to Earth.
- Mars Helicopter, **Ingenuity**, is a technology demonstration to test powered, controlled flight on another world for the first time.
- **Spirit and Opportunity (MER)**: These twin rovers landed on Mars in 2004 and explored the planet for over 14 years, far exceeding their original 90-day mission goals.
 - Good Night Oppy
- **Curiosity (MSL)**: This rover landed on Mars in 2012 and is still active today.
- **Artemis** is the name of NASA's program to return astronauts to the lunar surface. LTV?
- NASA's **CADRE** project – CADRE Robot
- **A-PUFFER**: foldable Autonomous Pop-Up Flat Folding Explorer Robot
- **DuAxel** is a prototype rover developed by NASA's Jet Propulsion Laboratory (JPL) to explore the toughest terrain on planetary surfaces, such as steep slopes, crater walls, and caves.





Differential Drive Robot

- **Structure**

- **Two powered wheels** (left & right) provide forward/backward motion and turning.
- **Two passive caster/support wheels** for balance and stability.
- Symmetric design with a fixed axle between driving wheels.
- Motion is driven by independent control of the left and right wheels.

- **Constraints**

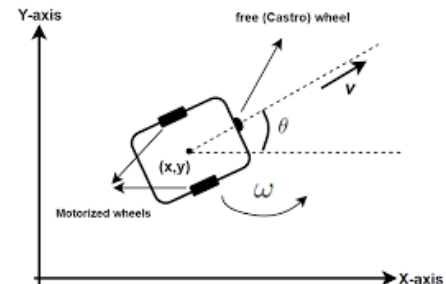
- **Nonholonomic constraint:** cannot move sideways.
- **Pure rolling assumption** (no lateral slip) for ideal motion.
- Errors arise due to **slip, sliding, rolling resistance** on uneven or low-friction terrain.

$$v = \frac{r}{2}(\omega_R + \omega_L), \quad \omega = \frac{r}{L}(\omega_R - \omega_L)$$

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega$$

- **Control**

- Controlled via:
 - **Linear velocity (v)** → forward/backward speed.
 - **Angular velocity (ω)** → turning rate.
- Common controllers: PID, trajectory tracking, pure pursuit.
- Low-level motor control: PWM signals or velocity commands.





DD Assumptions

- Model Assumptions:

- Pure Rolling and No Slipping: The wheels roll without slipping on the ground.
- Non-holonomic Constraint: The robot cannot move sideways directly; its linear velocity is always in the direction of its orientation.
- Fixed Wheel Radius and Track Width: The radius of the wheels (r) and the distance between the wheels (track width, l) are constant.

- Rolling Constraint

- The velocity of the wheel at the point of contact with the ground must be tangential to the wheel's direction.
- The robot can move forward or backward, but not sideways.

- Sliding Constraint

- The wheels must not slide in the direction perpendicular to their rolling direction.

- Slipping Constraint

- Assumes perfect traction and alignment. Robot moves straight with velocity v .





Kinematic Equations – Solved using Euler

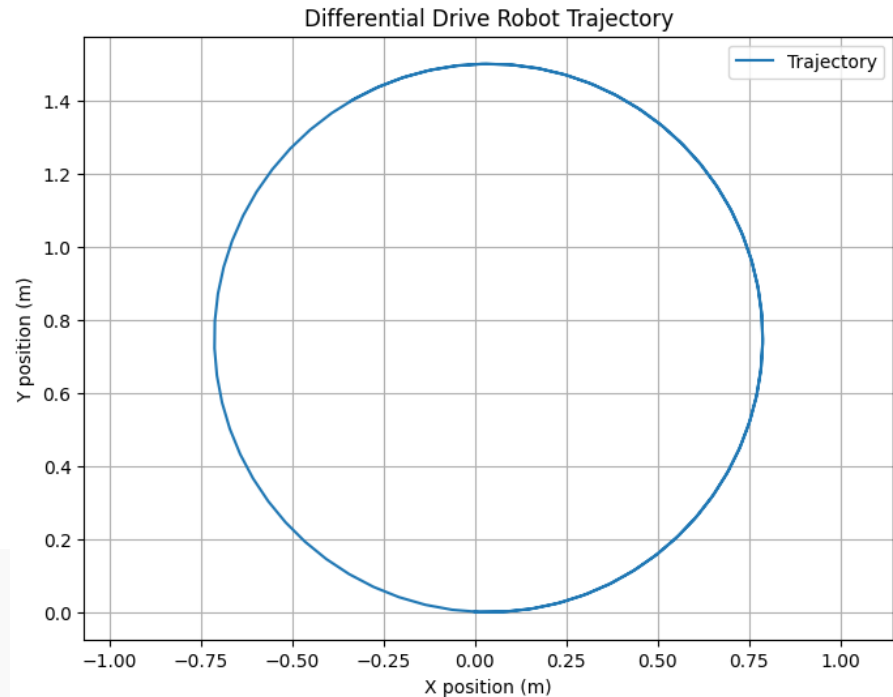
- To simulate the forward kinematics, these differential equations are integrated over time. A common approach is to use numerical integration, such as the Euler method or more sophisticated solvers (odeint).

Linear velocity: $v = \frac{v_r + v_l}{2}$

Angular velocity: $\omega = \frac{v_r - v_l}{\text{wheel_base}}$

Pose update using Euler integration:

- $x_{t+1} = x_t + v \cdot \cos(\theta_t) \cdot dt$
- $y_{t+1} = y_t + v \cdot \sin(\theta_t) \cdot dt$
- $\theta_{t+1} = \theta_t + \omega \cdot dt$



Start @ (0,0)

Wheel velocities (left and right)

$v_l = 0.5$ # m/s

$v_r = 1.0$ # m/s

Time step and total simulation time

$dt = 0.1$ # seconds

$\text{total_time} = 10.0$ # seconds





Forward kinematics of Ackermann steering

- Let the car have the shape of a box with length L between rear and front axis.
- Instantaneous center of rotation* (ICR) of the car is R .
- the steering angle φ is given by

$$\tan(\varphi) = \frac{L}{R}$$

differential drive

$$\frac{L}{R - l/2} = \tan(\pi/2 - \phi_r)$$

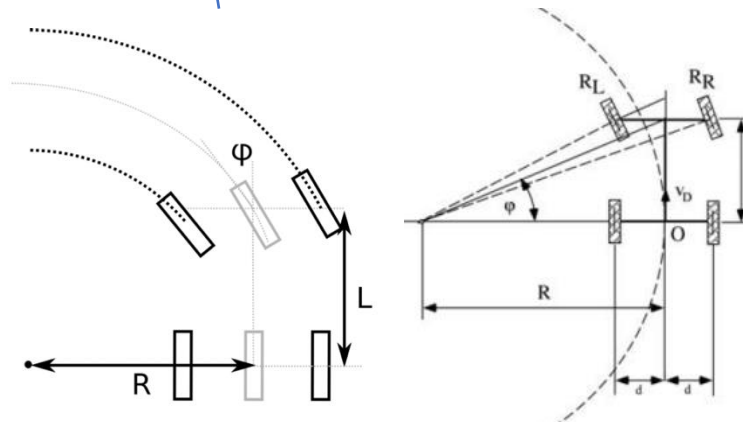
$$\frac{L}{R + l/2} = \tan(\pi/2 - \phi_l)$$

- angles of the left and the right wheel, φ_l and φ_r
- If not differential drive, $\varphi = \varphi_l = \varphi_r$
- Assuming the wheel speed to be $\dot{\omega}$ and the wheel radius r , we can calculate the speeds in the robot's coordinate frame to

$$\dot{x} = \dot{\omega} r$$

$$\dot{y} = 0$$

$$\dot{\theta} = \frac{\dot{\omega} r \tan \varphi}{L}$$





Forward kinematics of Ackermann steering

- The front wheels steer at an angle, causing the vehicle to follow a circular arc.
- The ICR lies along the rear axle, at a distance R , where:
 - L is the wheelbase.
 - δ is the steering angle.
- The angular velocity is $\omega = \frac{v}{R}$, and the vehicle rotates around the ICR.

- **Turning radius:**

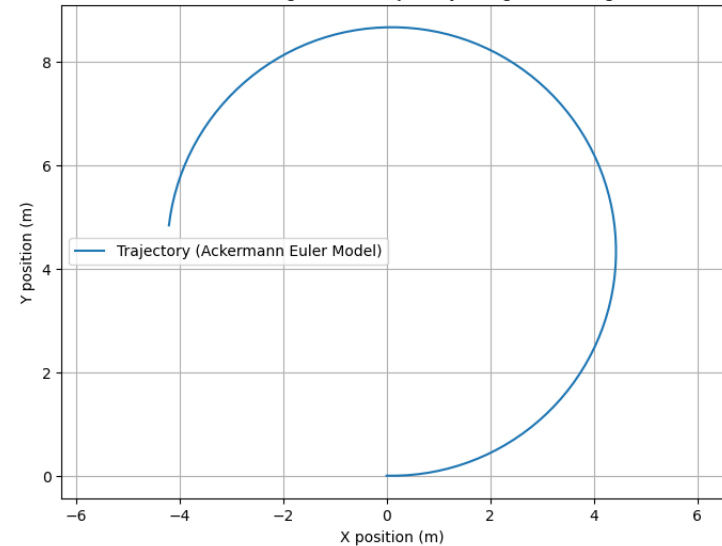
$$R = \frac{\text{wheel_base}}{\tan(\text{steering_angle})}$$

- **Angular velocity:** $\omega = \frac{v}{R}$

- **ICR coordinates:**

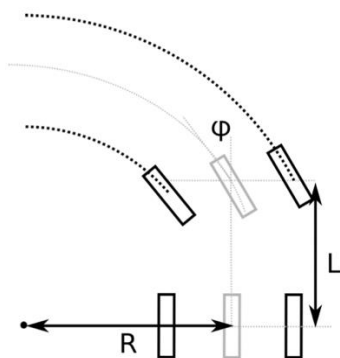
$$\begin{aligned} \text{ICR}_x &= x - R \cdot \sin(\theta) \\ \text{ICR}_y &= y + R \cdot \cos(\theta) \end{aligned}$$

Ackermann Steering Vehicle Trajectory using Euler Integration



Euler Equations

$$\begin{aligned} x_{t+1} &= x_t + v \cdot \cos(\theta_t) \cdot dt \\ y_{t+1} &= y_t + v \cdot \sin(\theta_t) \cdot dt \\ \theta_{t+1} &= \theta_t + \omega \cdot dt \end{aligned}$$





Forward kinematics - Differential Ackermann steering

- **Differential drive:** where left and right wheel velocities differ.
- **Ackermann steering:** where the vehicle turns based on steering geometry.

The **left and right front wheels** steer at different angles: δ_l and δ_r .

Each wheel defines its own turning radius:

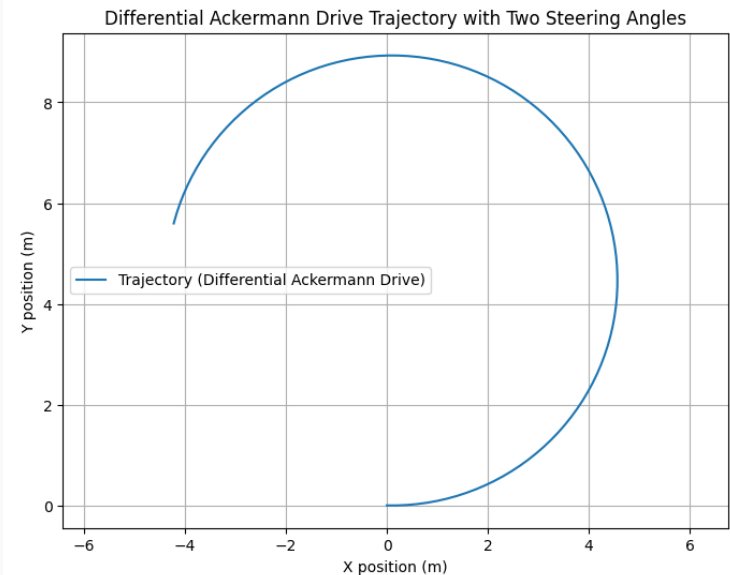
- $R_l = \frac{L}{\tan(\delta_l)}$
- $R_r = \frac{L}{\tan(\delta_r)}$

The **effective turning radius** is the average: $R_{ICR} = \frac{R_l + R_r}{2}$

The **angular velocity** is $\omega = \frac{v}{R_{ICR}}$

The pose is updated using Euler integration:

- $x_{t+1} = x_t + v \cdot \cos(\theta_t) \cdot dt$
- $y_{t+1} = y_t + v \cdot \sin(\theta_t) \cdot dt$
- $\theta_{t+1} = \theta_t + \omega \cdot dt$





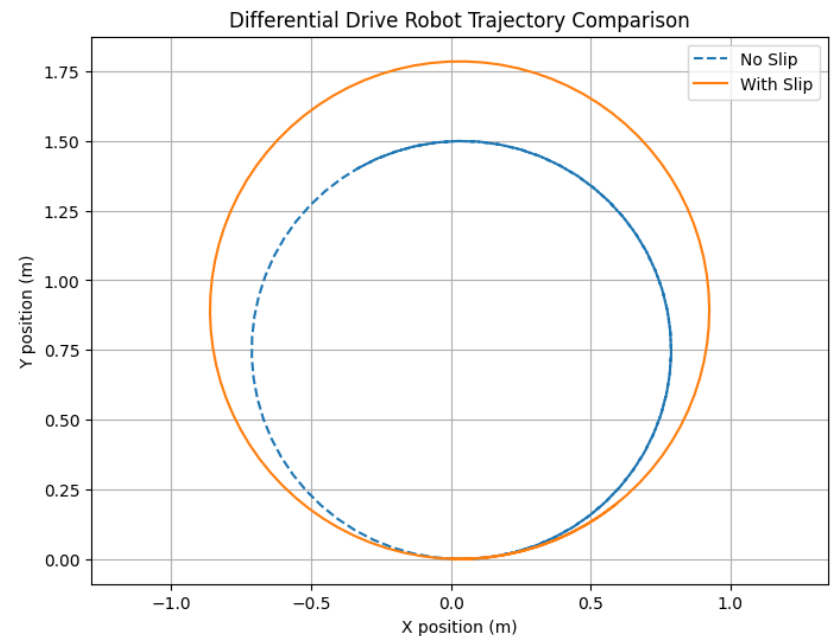
Differential drive with slip effects

- **Slip ratio** is the fraction of velocity lost due to slippage.
- For example, a slip ratio of 0.1 means the wheel effectively moves at 90% of its commanded speed.
- Effective wheel velocities:

$$v_l^{eff} = v_l \cdot (1 - \text{slip}_l)$$
$$v_r^{eff} = v_r \cdot (1 - \text{slip}_r)$$

- Linear and Angular velocities:

$$v = \frac{v_r^{eff} + v_l^{eff}}{2}$$
$$\omega = \frac{v_r^{eff} - v_l^{eff}}{\text{wheel_base}}$$



10% slip on the left wheel and 20% on the right

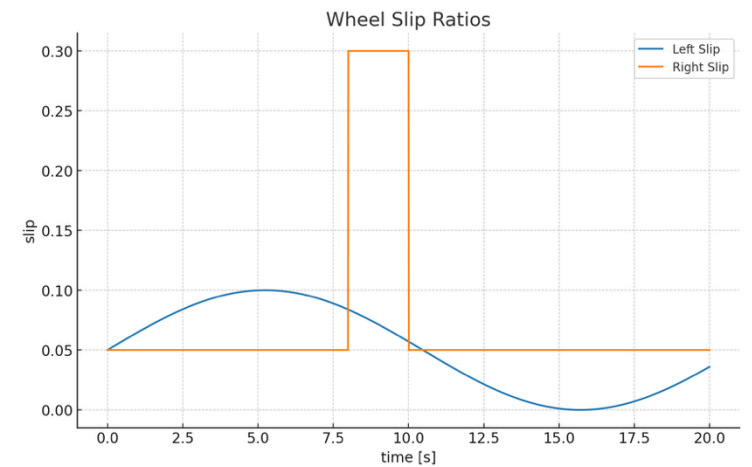
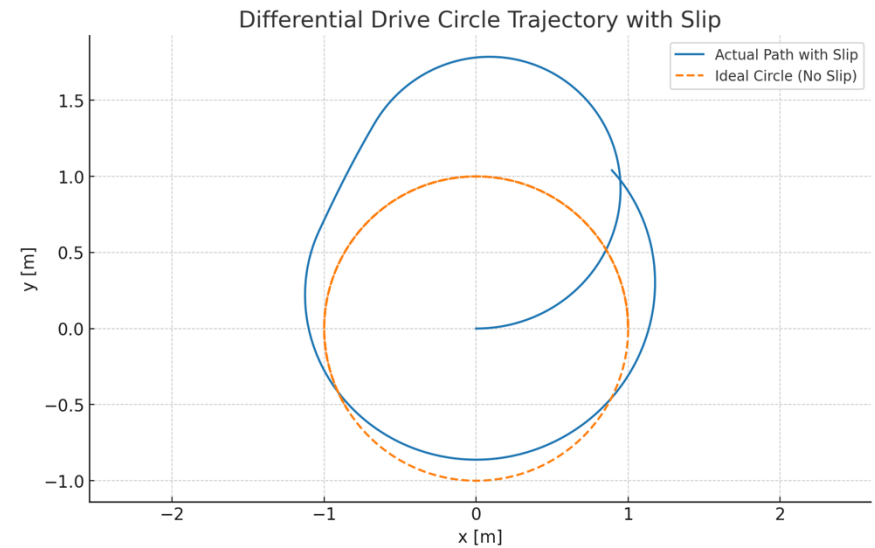
Slip causes the robot to deviate from its intended path - DRIFT





More on slips

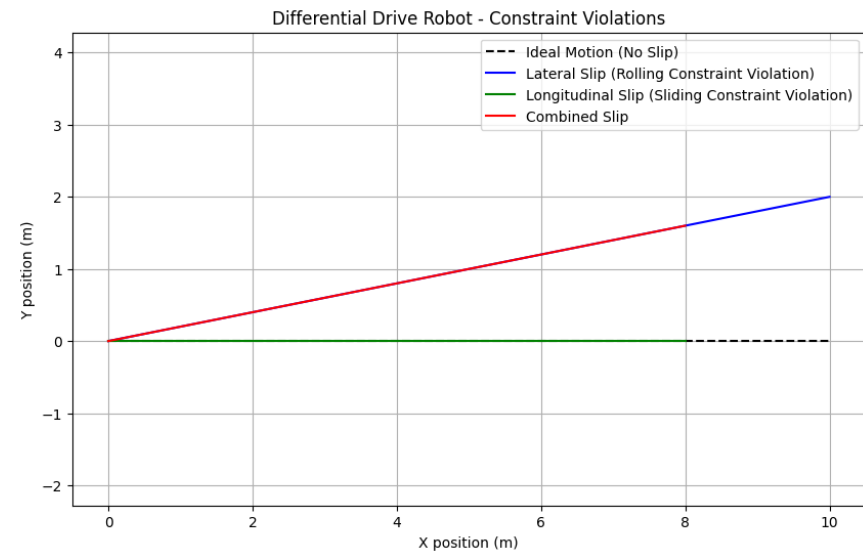
- Dynamic slip modeling is useful for simulating real-world conditions like:
 - Wet or icy surfaces
 - Uneven terrain
 - Tire wear or load imbalance
- Dynamic Slip:
 - Left slip oscillates slowly
 - Right slip with a short high-slip patch between 8s and 10s





Rolling & Sliding Constraint

- **Ideal Motion (Dashed Line)**
 - No constraint violations.
- Robot moves straight as expected.
 - **Lateral Slip (Rolling Constraint Violation)**
 - The robot experiences sideways drift.
- **Longitudinal Slip (Sliding Constraint Violation)**
 - The robot moves slower than commanded.
- **Parameters Used:**
 - **Wheel velocities:** 1.0 m/s (both wheels)
 - **Wheel base:** 0.5 meters
 - **Time step:** 0.1 seconds
 - **Total time:** 10 seconds
 - **Slip ratios:** 20% for lateral and longitudinal cases



- **Longitudinal slip** reduces effective velocity: $v_{eff} = v \cdot (1 - slip_{longitudinal})$
- **Lateral slip** adds drift perpendicular to heading: $y_{new} += slip_{lateral} \cdot v \cdot dt$





Rolling & Sliding Constraint

- **Ideal Motion (No Slip) – Dashed black line**

- Assumes perfect traction and alignment.
- Robot moves straight with velocity v .
- **Model:**

$$x_{t+1} = x_t + v \cdot \cos(\theta) \cdot dt$$

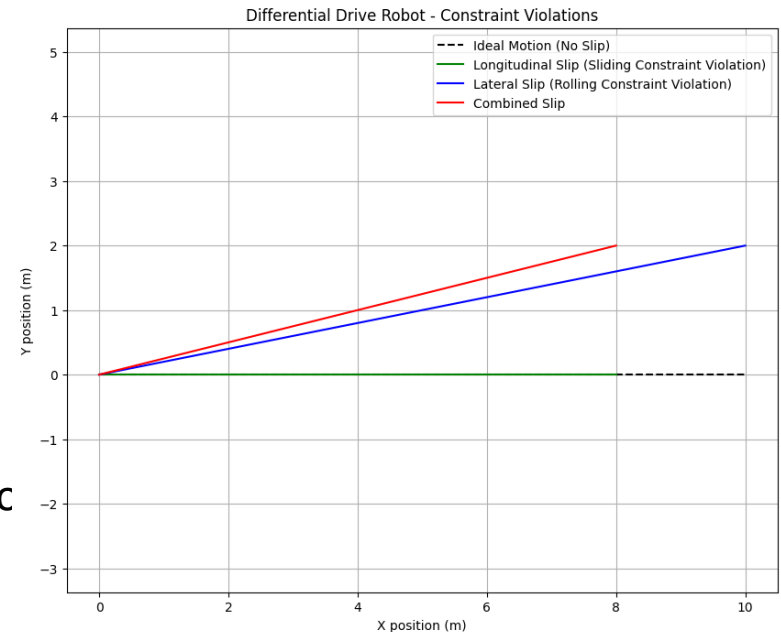
$$y_{t+1} = y_t + v \cdot \sin(\theta) \cdot dt$$

- **Longitudinal Slip (Sliding Constraint Violation) – Green line**

- Robot moves slower than commanded due to poor traction.
 - **Model:**
- $$v_{eff} = v \cdot (1 - slip_{longitudinal})$$

- **Lateral Slip (Rolling Constraint Violation) – Blue line**

- Robot drifts sideways due to imperfect wheel alignment or surface interaction.
 - **Model:**
- $$y_{t+1} += slip_{lateral} \cdot v \cdot dt$$



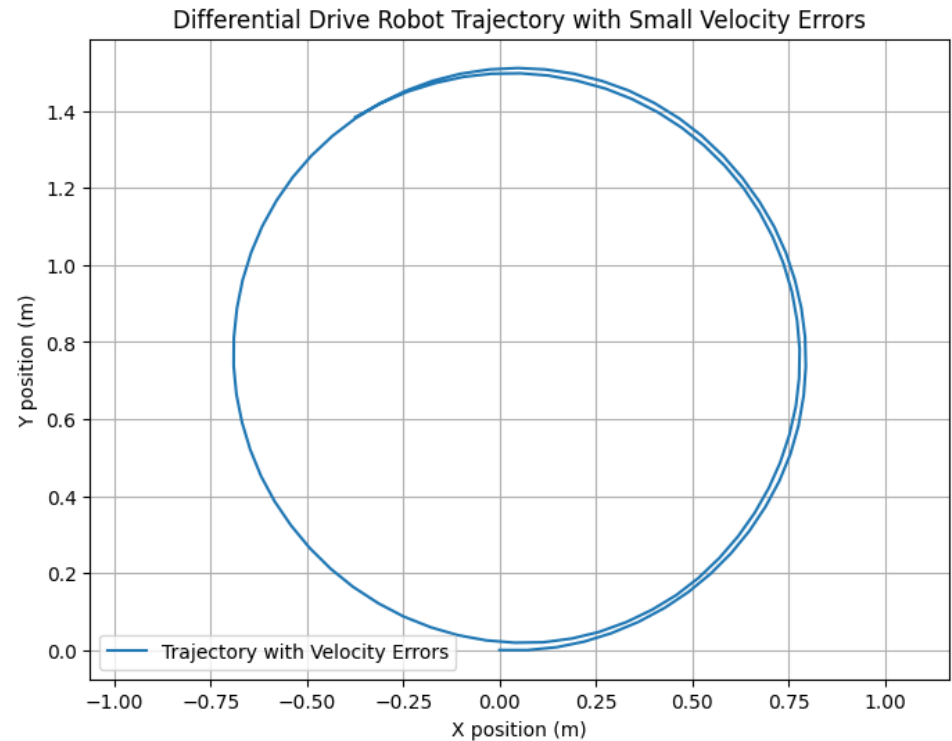
Both longitudinal and lateral slips are present.
Robot slows down and drifts off course.





Velocity Errors

- robot's trajectory deviates slightly from the ideal path due to **random noise** in the left and right wheel velocities.
- Simulation Example:
 - **Wheel velocities:** 0.5 m/s (left), 1.0 m/s (right)
 - **Wheel base:** 0.5 meters
 - **Error standard deviation:** 0.02 m/s
 - **Time step:** 0.1 seconds
 - **Total time:** 10 seconds





Error in wheel velocities – Euler

```
import numpy as np
import matplotlib.pyplot as plt

L = 0.2
sigma = 0.05

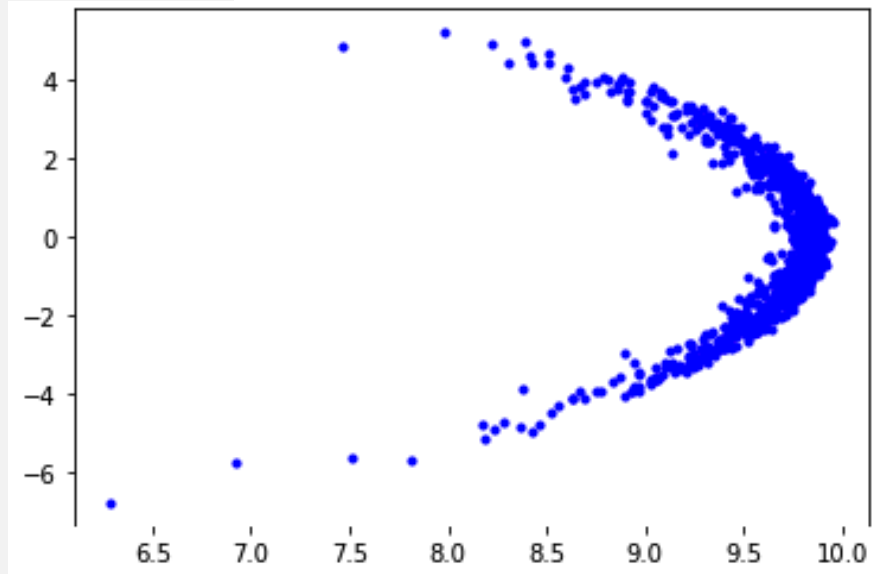
plt.figure()

for i in range(1000):
    vR = 1 + np.random.normal(0, sigma)
    vL = 1 + np.random.normal(0, sigma)
    theta = 0
    x = 0
    y = 0
    dt = 0.1

    for t in np.arange(0.1, 10, dt):
        x = x + 0.5 * (vR + vL) * np.cos(theta) * dt
        y = y + 0.5 * (vR + vL) * np.sin(theta) * dt
        theta = theta + 1 / L * (vR - vL) * dt
        vR = 1 + np.random.normal(0, sigma)
        vL = 1 + np.random.normal(0, sigma)

    plt.plot(x, y, 'b.')

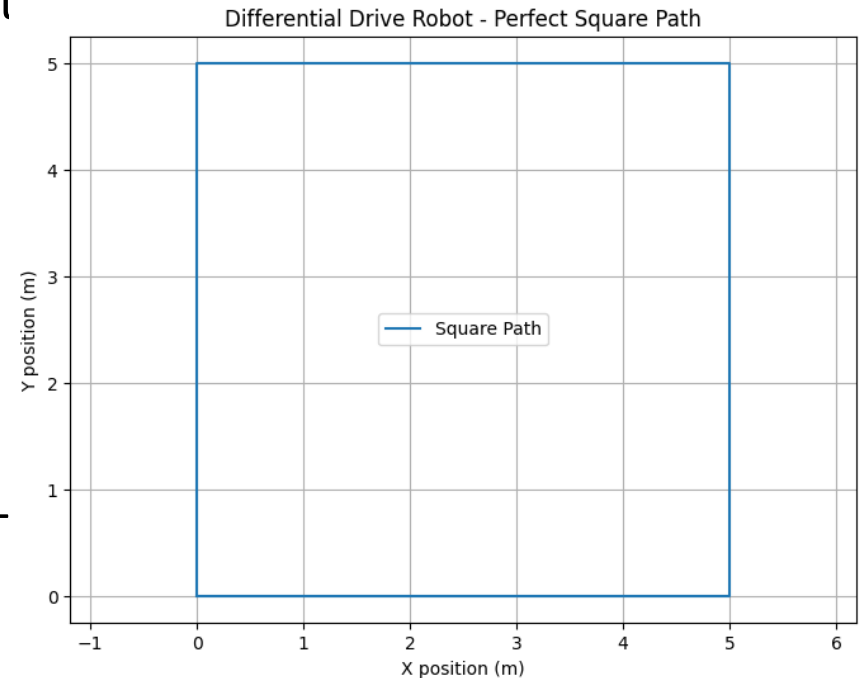
plt.show()
```





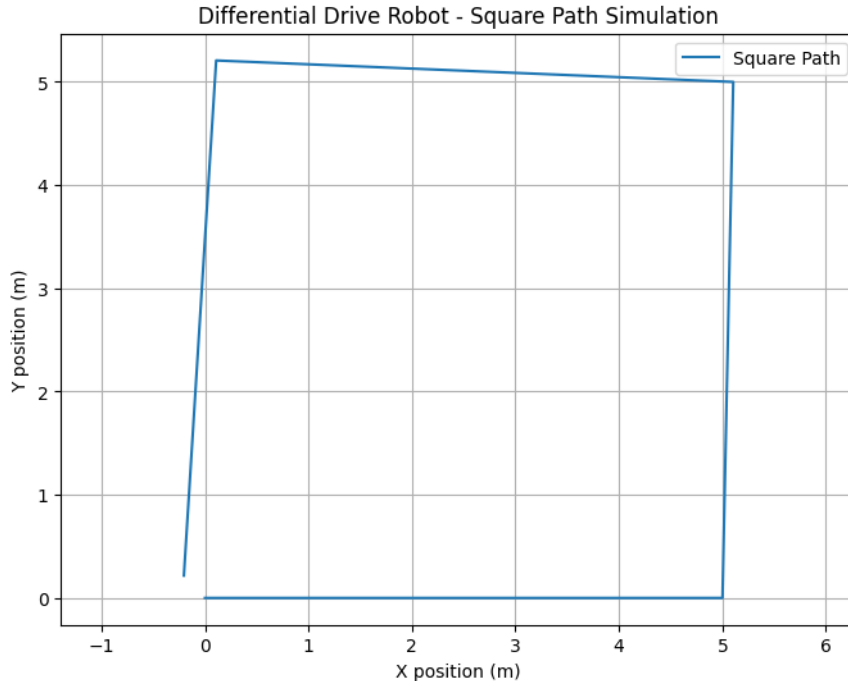
Differential drive robot - Square

- **Straight motion:**
 - Each side is traversed with constant linear speed for a fixed duration.
- **Precise rotation:**
 - At each corner, the robot performs an exact 90-degree in-place turn using a known angular speed and duration.
- **Consistent timing:**
 - Both motion and rotation are time-controlled to ensure accuracy.
- **Key Parameters:**
 - **Side length:** 5 meters
 - **Linear speed:** 1 m/s
 - **Angular speed:** $\pi/4$ rad/s (45°/s)
 - **Time step:** 0.1 seconds

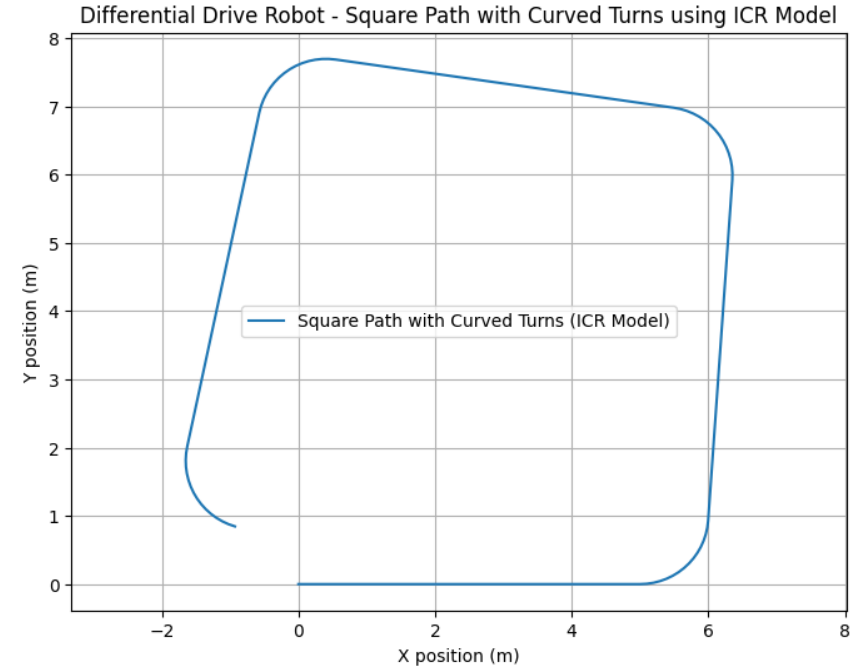




With errors in turn



- Side length: 5 meters
- Linear speed: 1 m/s
- Turn speed: 0.5 rad/s
- Time step: 0.1 seconds



- Turn radius: 1 meter
- Linear speed: 1 m/s
- Side length: 5 meters
- Time step: 0.1 seconds

trajectory shows slight deviations from a perfect square due to slip during rotation.





Fixing slipping, rolling, and sliding errors

- Requires a multi-layered approach addressing both hardware and software aspects:
- Hardware:
 - Ensure wheels are properly aligned and perpendicular to the axle
 - Check for wheel wear, flat spots, or imperfections, identical.
 - mounting is secure with no play or wobble.
 - Accurate 'r' and 'l' measurements





Fixing slipping, rolling, and sliding errors

- Software:
 - Straight line test and adjust for corrections
 - Validate straight-line distance tests over known distances
 - Execute rotation tests (360° turns) to verify angular accuracy.
- **Slip Detection and Compensation:**
 - Monitor encoder feedback vs. expected motion
 - Compare odometry with external position feedback (IMU, visual odometry)





Differential Drive Inverse Kinematics

- Given desired **linear velocity (v)** and **angular velocity (ω)** of the robot, find the **wheel angular velocities (ω_L, ω_R)**.

- Linear velocity of robot center:

$$v = \frac{r}{2} (\omega_R + \omega_L)$$

- Angular velocity of robot:

$$\omega = \frac{r}{L} (\omega_R - \omega_L)$$

- Inverse Kinematics (solve for wheels):

$$\omega_R = \frac{v}{r} + \frac{L}{2r} \omega$$

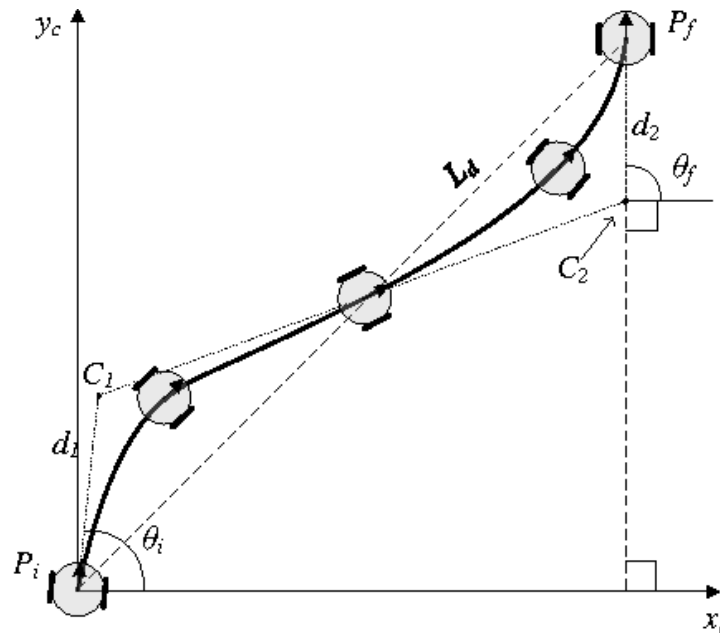
$$\omega_L = \frac{v}{r} - \frac{L}{2r} \omega$$





Use of IK? Goto Goal

- Path Planning (geometry)
- Trajectory Planning (Kinematics, time)
- Pose Estimation/Prediction using forward kinematics (dead reckoning)
- Pose regulation using Inverse Kinematics

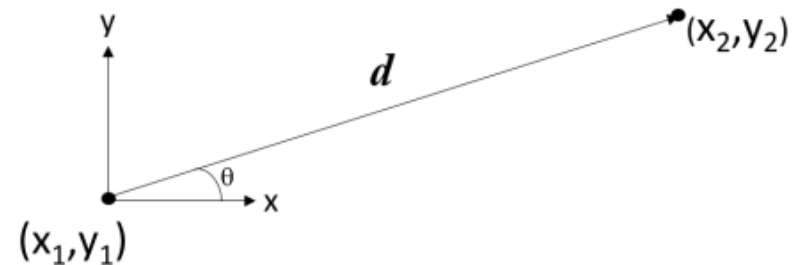




Steering to a point

- To steer the robot to desired position $[x_2, y_2]$ along a st. line is given as

$$\theta = \tan^{-1}\left(\frac{x_2 - x_1}{y_2 - y_1}\right)$$



$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

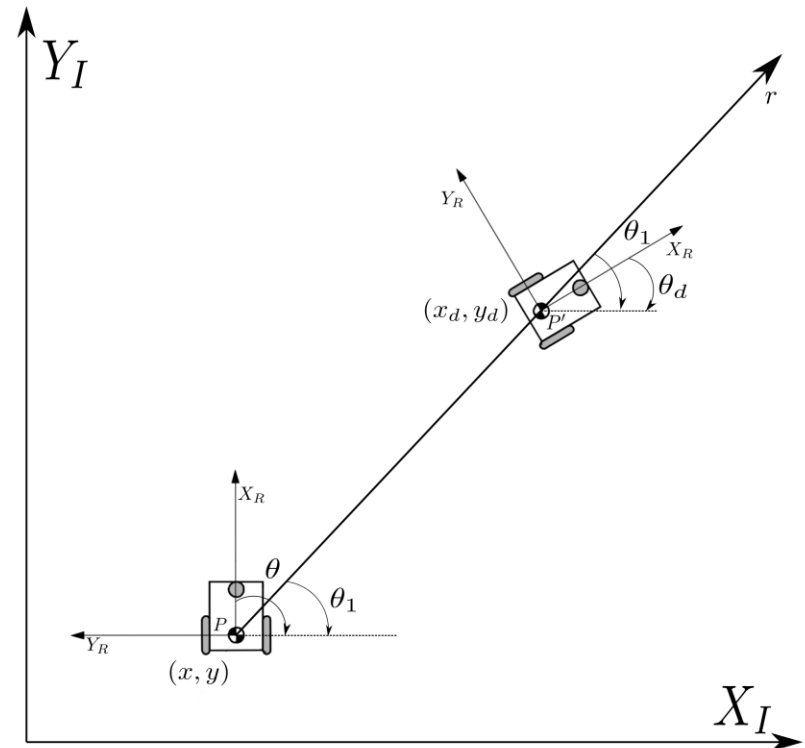
- How much 'time' do we need to spin and move forward to get to the desired pose?





Steering to a point - Simple Solution

- If we would like to drive the robot from any pose to some other pose in the global frame, we can decompose the motion in two rotations in place and one translation along a straight line.
- The robot can turn in the place aligning its orientation aiming the goal position, (x_d, y_d) , then move forward to the goal position, and then turn in the place again to reach the goal orientation, θ_d .
- The robot starts with the $\xi_i = [x, y, \theta]^T$. Then it spun around the point P and aim the desired position $P' = (x_d, y_d)$ reaching the pose $\xi'_i = [x, y, \theta_1]^T$. To reach the position, it moves forward to $P' = (x_d, y_d)$ and reaches $\xi''_i = [x_d, y_d, \theta_1]^T$. And then the robot spun again to from θ_1 to θ_d . The final robot state should be $\xi'''_i = [x_d, y_d, \theta_d]^T$.



Still the main question is 'what is 't'' for every step/motion!





Steering to a point - Rotation

- To **rotate about the same point** (no translation), set the linear speed to zero:

$$v = \frac{r}{2} (\omega_R + \omega_L) = 0 \Rightarrow \omega_R = -\omega_L.$$

- Inputs:

- $L=0.30$ m, $r = 0.05$ m, $\omega_{wheel,max} = 8$ rad/s
- $\omega_{max} = \frac{2r}{L} \omega_{wheel,max} = \frac{0.10}{0.30} \cdot 8 \approx 2.67$ rad/s
- Rotate $90^\circ \Rightarrow \Delta\theta = \pi/2$

- Simple controller (constant rate spin)

- Compute wrapped angle error $\Delta\theta \in [-\pi, +\pi]$.
- Pick $\omega = \lceil (k_p \cdot \Delta\theta, -\omega_{max}, +\omega_{max}) \rceil$.
- Send wheel commands $\omega_R = \frac{L}{2r} \omega$, $\omega_L = -\frac{L}{2r} \omega$.
- **Best-case time** (speed-limited): $T_{min} = \frac{\pi/2}{2.67} \approx 0.59$ s.
- Stop when $|\Delta\theta| < \epsilon$.





Steering to a point - distance

- **Compute the line to the goal**

- $\Delta x = x_2 - x_1, \Delta y = y_2 - y_1,$
- $d = \sqrt{\Delta x^2 + \Delta y^2}, \alpha = \text{atan2}(\Delta y, \Delta x)$

- **Rotate in place** to face the line: align to α (from θ_1)

- **Drive straight** the distance d (keep $\omega = 0$):

- $v = \text{chosen constant forward speed}, \omega = 0 \Rightarrow \omega_R = \omega_L = \frac{v}{r}$
- Translate for time $T_{lin} = \frac{d}{v}$.

- **Rotate in place** to face the final orientation: align to θ_2

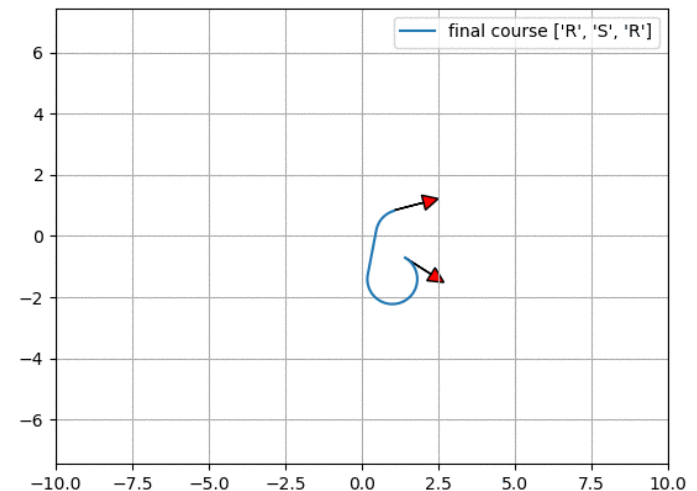
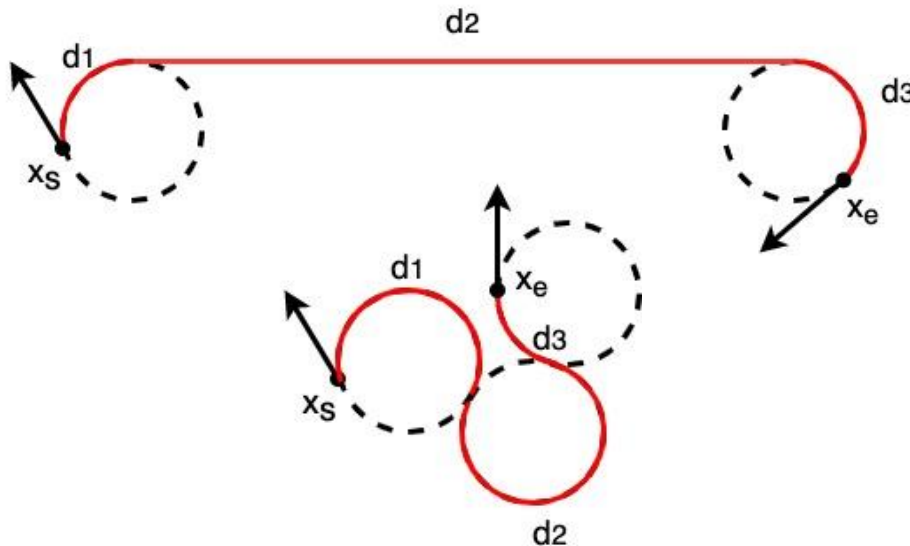
- The total time is
$$T = \frac{2|\Delta\theta|}{|\omega|} + \frac{d}{v}$$





Dublin Path Algorithm

- Dublin Path algorithm is a motion planning algorithm that generates a smooth path between two points while minimizing curvature.
- Use moves such as RSR, LSL, RSL, ...



- real-world implementation, you'll likely need a control loop to ensure that the robot follows the path accurately.





Dublin Path Algorithm

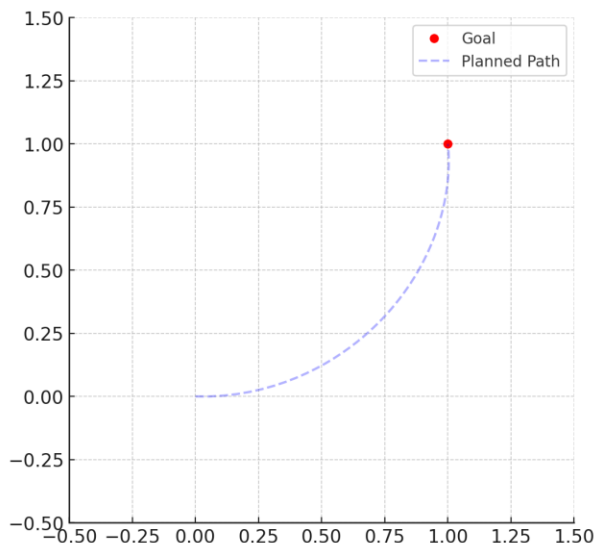
- There are **six possible Dubins paths**, each a sequence of **Left turn (L)**, **Straight (S)**, and **Right turn (R)** arcs:
 - LSL, LSR
 - RSL, RSR
 - RLR, LRL
- Example:
 - **LSL** → Left arc → Straight → Left arc.
 - **RSR** → Right arc → Straight → Right arc.
- The algorithm checks all six candidates and picks the shortest feasible one.
- Steps to move the robot with Dubins path:
 - **Input:** start pose (x_1, y_1, θ_1) , goal pose (x_2, y_2, θ_2) , minimum turning radius R .
 - **Compute candidate paths** (LSL, LSR, RSL, RSR, RLR, LRL).
 - **Choose shortest path.**
 - **Generate wheel velocities** (via inverse kinematics) to follow the arcs and straight segments.
 - Execute in sequence:
 - Arc with angular velocity $\omega = \pm \frac{v}{R}$.
 - Straight with constant v .
 - Final arc to match orientation.





Simultaneous Distance & Angle Adjustment

- **go-to-goal controller** for a differential-drive robot.
- At each timestep it:
 - measures the **distance** and **bearing** to the goal position,
 - commands a **linear speed** proportional to distance and an **angular speed** proportional to heading error,
 - converts (v, ω) to **wheel angular speeds** via inverse kinematics,
 - integrates the forward kinematics to update (x, y, θ) .
 - This repeats until the robot is close enough to the goal.



- Inputs
 - r (wheel radius), L (wheel separation), dt (integration step).
 - Start pose (x, y, θ) , goal pose (x_g, y_g, θ_g) .
 - Gains: k_dist , $k_heading$.
- Error computation
 - $dx = x_goal - x$
 - $dy = y_goal - y$
 - $dist = \text{np.hypot}(dx, dy)$
 - $target_heading = \text{atan2}(dy, dx)$
 - $heading_error = \text{wrap}(target_heading - \theta, -\pi, \pi)$
- Control Law:
 - $v = k_dist * dist$
 - $\omega = k_heading * heading_error$
- Inverse kinematics $\rightarrow$ wheel speeds
- Forward kinematics integration (Euler) $\rightarrow$ Update position and angle.





Polar Controller

- Simultaneous Distance & Angle Adjustment (Polar Controller)
- Define errors in polar form:

$$\rho = \sqrt{(x_2 - x)^2 + (y_2 - y)^2},$$

$$\alpha = (\arctan 2(y_2 - y, x_2 - x) - \theta) [-\pi, +\pi]$$

$$\beta = (\theta_2 - \theta - \alpha) [-\pi, +\pi].$$

- Control law (choose gains with $k_\rho > 0$, $k_\alpha > 0$, $k_\beta < 0$, and typically $k_\alpha > k_\rho$):

$$v = k_\rho \rho, \omega = k_\alpha \alpha + k_\beta \beta$$

- This drives $\rho \rightarrow 0$ and $\theta \rightarrow \theta_2$ at the same time (no stop-go phases).
- Convert to wheel speeds (track L , wheel radius r):

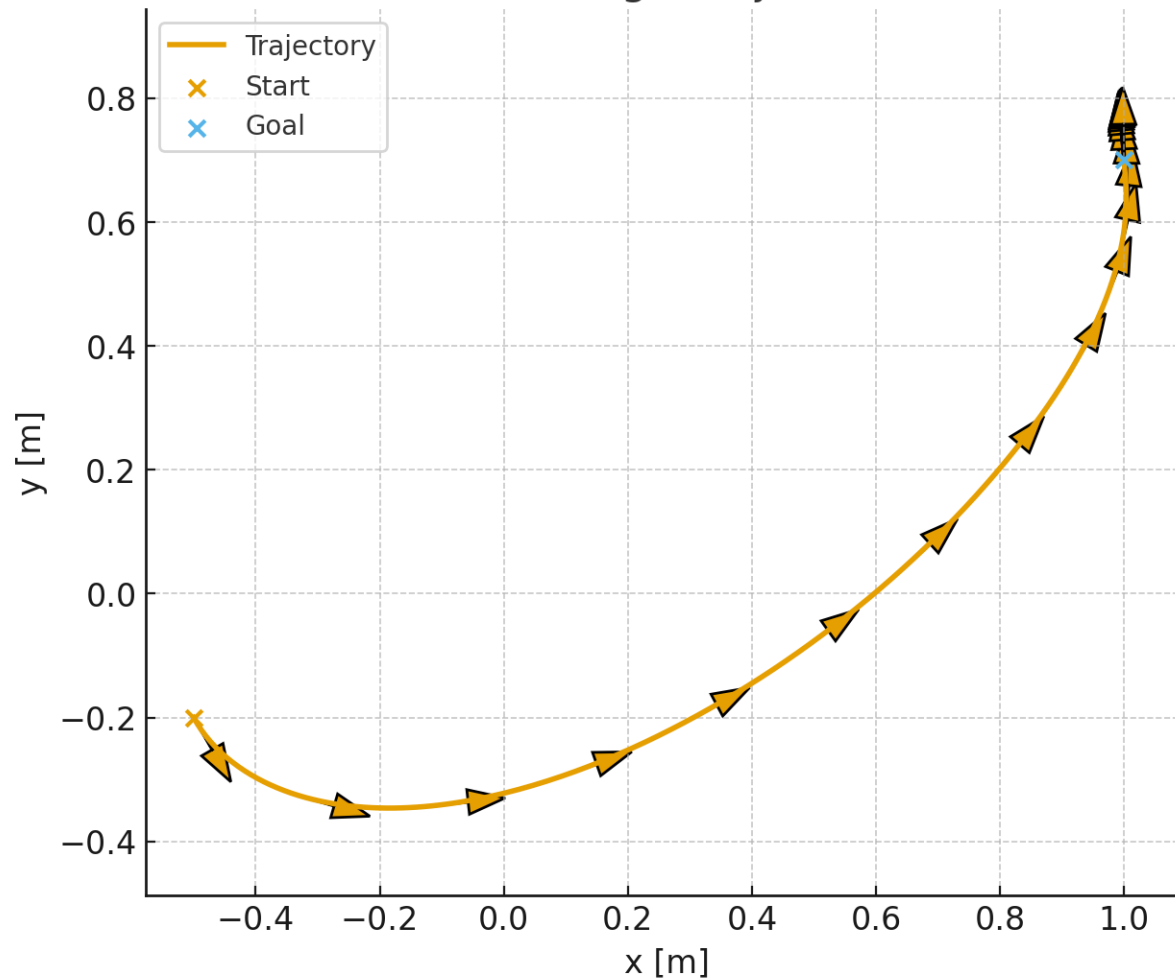
$$v_R = v + \frac{L}{2} \omega, v_L = v - \frac{L}{2} \omega, \omega_R = \frac{v_R}{r}, \omega_L = \frac{v_L}{r}.$$





Polar Controller

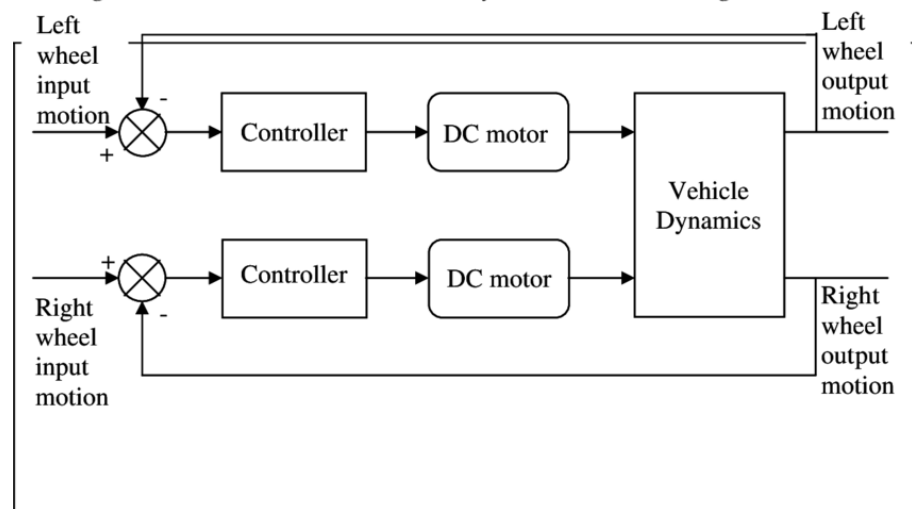
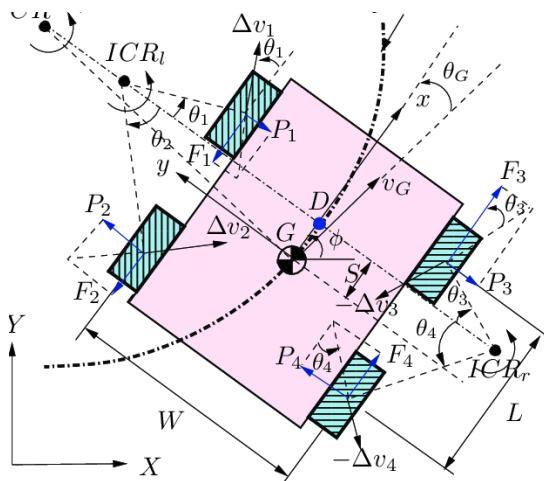
Simultaneous Distance & Angle Adjustment (Polar Controller)





What is Dynamics?

- *Dynamic models* describes
 - system motion when forces are applied to the system - The effect of all forces (internal and external) on a robot's motion.
 - physics of motion where forces, energies, system mass, inertia, and velocity parameters - **If the motors apply this much force, where will the robot be?**
 - Expressed as second order differential equations





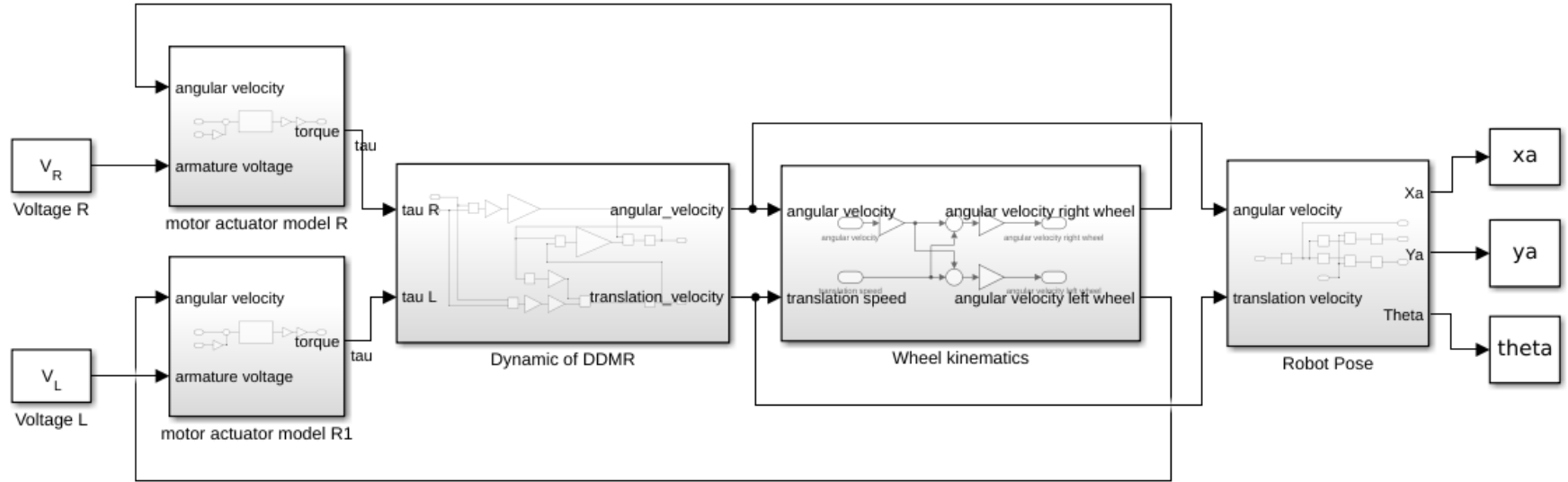
Why Dynamics?

- Goal: to get our robot to (x_g, y_g, θ_g)
- Process:
 - find/estimate current location; path planning; trajectory planning; pose execution (control)
- Questions:
 - Is the kinematic model sufficient for control design?
 - Do we need dynamics for control?
 - Dynamics is the study of the motion of a mechanical system taking into consideration the different forces that affect its motion





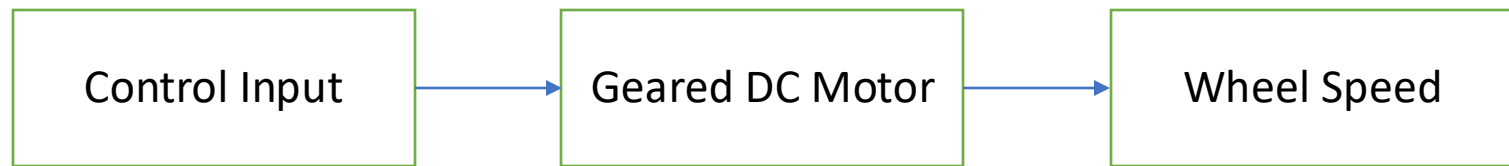
A complete model of DDMWR on Simulink





Mobile Robot Dynamics

- Dynamics explains the relationship b/w control input (u) and wheel velocity output ($\dot{\phi}_L, \dot{\phi}_R$)



- From classical mechanics, the sum of the torques equals the product of the moment of inertia and angular acceleration,

$$\Sigma\tau = \mathfrak{I}\alpha.$$

- Where $\Sigma\tau$ = sum of torques, $\mathfrak{I}$ product of moment of inertia, and α angular acceleration.





- Newton's Second Law for Rotation:

$$\text{Sum of Torques } (\Sigma \tau) = \text{Moment of Inertia } (I) \times \text{Angular Acceleration } (\alpha)$$

1. Torque (τ) is the rotational equivalent of force in linear motion. It is the measure of the tendency of a force to rotate an object.
2. Moment of Inertia (I) is a measure of an object's resistance to rotational motion and depends on both the mass of the object and how that mass is distributed relative to the axis of rotation.
3. Angular Acceleration (α) represents the rate of change of angular velocity, describing how quickly an object is speeding up or slowing down in its rotation.





Mobile Robot Dynamics

- Expanding that concept to represent all the mechanical torques acting on the robot

$$\mathbf{M}\dot{\boldsymbol{\omega}} + \mathbf{C}\boldsymbol{\omega} + \mathbf{F}\boldsymbol{\omega} + \mathbf{G} = \mathbf{B}\boldsymbol{\tau} - \mathbf{A}\boldsymbol{\lambda}$$

- **M** is the matrix of inertial moments, **C** is the matrix of Coriolis/centrifugal forces, **F** is the matrix of frictional forces, **G** is the gravitational vector, **B** describes the transformation/application of input torques and **A** describes the kinematic constraints of the platform.

- simplified expression of the above expression

$$\mathbf{M}\dot{\boldsymbol{\varphi}} + \mathbf{F}\boldsymbol{\varphi} = \mathbf{B}\boldsymbol{\tau}$$

- **τ** in equation the above equation represents the mechanical torque input vector developed by the electric motors.





Mobile Robot Dynamics

- Electrical Dynamics in DC Motors
- Electric Torque:

$$\tau = K_t I_a$$

- KVL of the DC Motor model:

$$-V_{dc} + I_a R_a + L_a \dot{I}_a + E_b = 0$$

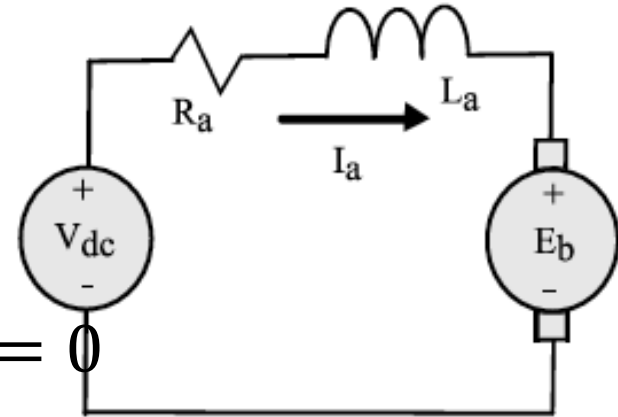
- Back EMF:

$$E_b = K_a \varphi$$

$$I_a R_a + L_a \dot{I}_a = -K_a \varphi + V_{dc}$$

- equate the electrical torque to the mechanical torque under equilibrium conditions

$$T_m = \mathfrak{I} \dot{\varphi} + F \varphi = T_e = K_t I_a$$
$$\mathfrak{I} \dot{\varphi} + F \varphi = K_t I_a$$





Mobile Robot Dynamics

- In Laplace Domain:

$$\begin{aligned}(\mathfrak{I}s + F)\Phi(s) &= K_t I_a(s) \\ (L_a s + R_a)I_a(s) &= -K_a \Phi(s) + V_{dc}\end{aligned}$$

- Open-loop transfer function:

$$\frac{\Phi(s)}{V_{dc}} = \frac{K_t}{(\mathfrak{I}s + F)(L_a s + R_a) + K_t K_a}$$

- Each motor has two state variables, an armature current, I_a , and a shaft/wheel angular velocity, φ , as well as an input variable, V_{dc}
- Will represent state space model in the form,
 $\dot{\mathbf{x}} = \mathbf{Ax} + \mathbf{Bu}$ and $\dot{\mathbf{y}} = \mathbf{Cx}$





Mobile Robot Dynamics

The net result of forward motion is shown as:

$$\begin{bmatrix} \dot{I}_{a1} \\ \dot{\varphi}_1 \\ \dot{I}_{a2} \\ \dot{\varphi}_2 \\ \dot{v} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \frac{-R_{a1}}{L_{a1}} & \frac{-K_{a2}}{L_{a1}} & 0 & 0 & 0 & 0 \\ \frac{K_{t1}}{\mathfrak{I}_1} & \frac{-F_1}{\mathfrak{I}_1} & \frac{-R_{a1}}{L_{a2}} & \frac{-K_{a2}}{L_{a2}} & 0 & 0 \\ 0 & 0 & \frac{L_{a2}}{K_{t2}} & \frac{-F_2}{\mathfrak{I}_2} & 0 & 0 \\ 0 & 0 & \mathfrak{I}_2 & \mathfrak{I}_2 & 0 & 0 \\ 0 & r_1/2 & 0 & r_1/2 & 0 & 0 \\ 0 & r_1/2L & 0 & -r_2/2L & 0 & 0 \end{bmatrix} \begin{bmatrix} I_{a1} \\ \varphi_1 \\ I_{a2} \\ \varphi_2 \\ v \\ \theta \end{bmatrix} + \begin{bmatrix} \frac{V_{dc1}}{L_{a1}} \\ 0 \\ \frac{V_{dc2}}{L_{a2}} \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Robot Pose in Inertia Frame (non-linear):

$$\dot{\xi}_I = R(\dot{\theta}) \begin{bmatrix} \dot{v} \\ 0 \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{r(\varphi_1 + \varphi_2)}{2} \\ 0 \\ \frac{r(\varphi_1 - \varphi_2)}{2} \end{bmatrix}$$

System Output (position in Inertia frame)

$$Y = \xi_I = \begin{bmatrix} x_R \\ y_R \\ \theta_R \end{bmatrix}$$

$$\varphi_R \varphi_L \triangleq \varphi_1 \varphi_2$$





Physical Measurements of the DDMR

Parameter	Measurement Method	Value
Mass, m	gram scale	405 grams
Wheel Radius, ($r_1 = r_2$)	caliper	32.2 mm
Wheel Offset, L	tape measure	64 mm
Armature Resistance, R_{a1}	ohm-meter	5.32 ohms
Armature Resistance, R_{a1}	ohm-meter	5.41 ohms
Armature Reactance, L_{a1}	L-C meter	0.023 H
Armature Reactance, L_{a2}	L-C meter	0.024 H
Wheel RPM, No Load	tachometer	210 RPM
Gear Ratio, GR	Observed	48:1

The Result with Feedback Control considering dynamics

